

Summary of Neural Network Design Principles

6.2.3 Conditional Computation and Output Distributions

The output layer of a neural network models a conditional probability distribution ($p(\mathbf{y} \mid \mathbf{x}; \boldsymbol{\theta})$). The cost function is derived from the negative log-likelihood of this distribution, allowing flexibility for various output types (e.g., Gaussian, mixture models).

Example

- **Gaussian Output:** Predict both the mean and variance of a Gaussian distribution for regression tasks.
- **Mixture Density Networks:** Use a Gaussian mixture model to predict multiple modes in the output distribution, such as in speech synthesis or motion prediction.

Author's Message

- Use maximum likelihood to define cost functions for flexible output distributions.
- For complex outputs (e.g., multimodal data), use mixture models or heteroscedastic approaches.
- Numerical stability is crucial; avoid division by zero or unstable gradients.

6.3 Hidden Units

Hidden units transform inputs non-linearly, enabling neural networks to learn complex patterns. The choice of activation function (e.g., ReLU, sigmoid, tanh) significantly impacts training dynamics and performance.

Example

- **ReLU:** ($g(z) = \max\{0, z\}$), widely used for its simplicity and effectiveness.
- **Sigmoid/Tanh:** Saturating functions, historically popular but now less common due to vanishing gradients.

Author's Message

- **ReLU** is the default choice for hidden units due to its linear behavior and ease of optimization.
- Avoid saturating functions (e.g., sigmoid) in hidden layers unless necessary.
- Experimentation is key; no single activation function is universally best.

6.3.1 Rectified Linear Units (ReLUs) and Generalizations

ReLUs and their variants (e.g., Leaky ReLU, PReLU, Maxout) address the limitations of traditional saturating units by introducing non-saturating, piecewise-linear behavior.

Example

- **Leaky ReLU:** Allows small negative gradients: ($g(z) = \max\{0, z\} + \alpha \min\{0, z\}$)
- **Maxout:** Learns piecewise linear functions by taking the maximum of groups of inputs: ($g(\mathbf{z})_i = \max_{j \in \mathbb{G}(i)} z_j$).

Author's Message

- ReLUs are preferred for their simplicity and effectiveness.
- Generalizations like Leaky ReLU and Maxout can improve performance in specific cases.
- Non-differentiability at zero is not a practical issue for optimization.

6.3.2 Logistic Sigmoid and Hyperbolic Tangent

Sigmoid and tanh were historically popular but suffer from saturation, which hampers gradient flow. Tanh is slightly better due to its symmetry around zero.

Example

- **Sigmoid:** ($g(z) = \sigma(z)$), outputs probabilities (0 to 1), used in binary classification.
- **Tanh:** ($g(z) = \tanh(z)$), outputs values between -1 and 1, used in recurrent networks.

Author's Message

- Avoid sigmoid/tanh in hidden layers due to saturation and vanishing gradients.
 - Tanh is preferable to sigmoid if a saturating function is necessary.
-

6.3.3 Other Hidden Units

Essence

Less common but sometimes useful activation functions, such as softplus, radial basis functions (RBF), and hard tanh, offer alternatives for specific tasks.

Example

- **Softplus:** ($g(a) = \log(1 + e^a)$), smooth approximation of ReLU, but empirically less effective.
- **RBF:** ($h_i = \exp\left(-\frac{1}{\sigma_i^2} \|\mathbf{W}_{:,i} - \mathbf{x}\|^2\right)$), useful for modeling similarity to prototypes.

Author's Message

- Many activation functions work well, but ReLU and its variants are generally superior.

- Experimentation is encouraged, but new units must demonstrate clear advantages to be noteworthy.
-

6.4 Architecture Design

Neural network architecture involves organizing units into layers and defining their connectivity. The chain structure (feedforward) is most common, but other architectures (e.g., recurrent, convolutional) are used for specific tasks.

Example

- **Feedforward Network:** Layers connected sequentially, e.g.,
$$[\mathbf{h}^{(2)} = g^{(2)} (\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)}) .]$$
- **Recurrent Networks:** Used for sequential data, where outputs depend on previous states.

Author's Message

- Architecture design is problem-specific; experiment with layer types and connectivity.
- The choice of activation functions and layer organization impacts performance and training dynamics.

Summary of Author's Overarching Message

- **Flexibility:** Neural networks can model complex distributions and functions.
- **Practicality:** ReLU and its variants are default choices for hidden units; avoid saturating functions unless necessary.
- **Experimentation:** No one-size-fits-all solution; trial and error are essential in designing networks.
- **Stability:** Numerical stability and gradient flow are critical for successful training.